

AREA SCIENCE PARK
RESEARCH AND TECHNOLOGY INSTITUTE
LABORATORY OF DATA ENGINEERING

Web Application for DPCfam and DPCstruct Data Exploration

Report 8 (3 March - 9 Mar 2026) : Web Development
(DPCStruct Backend)

Emmanuel Nyandu Kagarabi
Master in Data Management and Curation (MDMC - SISSA)

Supervisors :
Valerio Piomponi
Elaheh Saadat

March 7, 2026

1 Introduction

This report is the eighth in a sequence documenting the weekly assigned tasks of our internship. Focusing on the DPCStruct dataset ([Barone et al., 2025](#)), we present the second model (`DpcStructMcsSequence`), which allows users to explore the full list of protein domains per metacluster and inspect a maximum of two representative sequences within that list. Likewise, the project has been updated so that users can select DPCStruct from the landing page, enter a valid metacluster ID, and, if everything goes well, explore a list of tables: Properties, Sequences, Downloads (`MCID.fasta` and `MCID.pdb`; each of the two files contains a maximum of two representatives: the cluster centre or the highest pLDDT representative). We will extend it soon with information on CATH/SCOP and a 3D Viewer (using `Mol*`). The page Downloads/DPCStruct offers the possibility to download two files containing all PDB and FASTA files.

2 Development

2.1 Database Creation in PostgreSQL

The files `dpcstruct_classification.tsv` and `mcs_reps.fasta` were preprocessed using the `dpcstruct_mcs_sequences_info.ipynb` notebook, where interesting outputs are : First, for each MCID, we collect its representatives (maximum of 2 : One representing the centre of the cluster and the other the domain with the highest pLDDT, in some cases, they are the same; resulting in one file (e.g.: MC26007, MC43377, MC61048, MC31933, MC50574,...). Second and finally, the two files were converted into dataframes and subsequently merged into a consistent one : `dpcstruct_mcs_sequences.csv` (Figure 1):

	mc_id	protein_id	prot_range	prot_seq
0	MC0	A0A1Q3ZAL7	4-118	LIFLVLTTTSFTTNLFKAQVNVNININSQPEWGPRGYNYVESYLL...
1	MC0	A0A537IMV5	3-112	KIFICFALCLLAFFKPASAQFSTNENIKDQPKWGLAGQKYVEYYY...
2	MC0	A0A170YWQ7	2-107	NaN
3	MC0	A0A3E2NVG7	1-133	NaN
4	MC0	A0A2W4V3S2	3-113	NaN
5	MC0	A0A4R2EMX1	49-158	NaN
6	MC0	A0A4R2EPD2	219-328	NaN
7	MC0	A0A533BJV2	138-258	NaN
8	MC0	A0A4Q3BL62	5-101	NaN
9	MC0	A0A161LV53	39-155	NaN

Figure 1: DPCStruct Metaclusters Domains : Full list of protein domains, but only representative sequences (amino acids) are available and, therefore, displayed in the last column.

Returning to our routine, we created a table in PostgreSQL with fields matching the columns of our dataframe. That operation was completed using `create_dpcstruct_tables.sql` script. The table was then populated with data from the CSV file, using `populate_dpctruct_tables.sql` script.

2.2 Django Routine

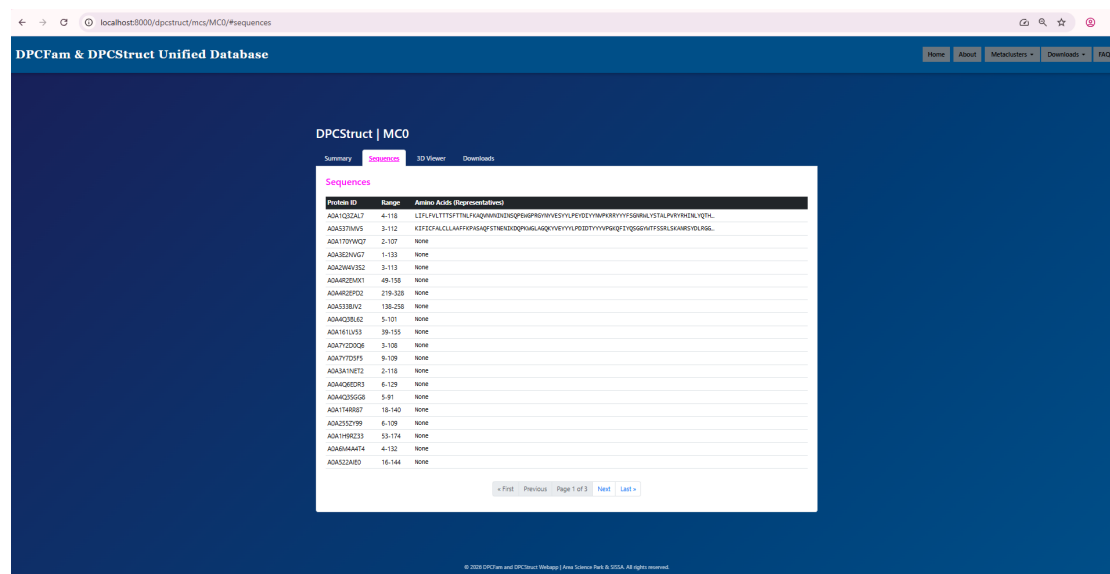
Following our routine, discussed in Report3 and Report7, we adhered to the standard

Model (M)→View (V)→Template (T)

pattern, updated the corresponding files, defined URLs, and mapped them to relevant views. Putting everything together, check out the `dpctruct` app.

Moving on, we updated the global (project) `views.py` in such a way that it handles requests from the landing page, including dpctruct. Users can now choose DPCstruct as the database from the landing page and enter a valid MCID. If it exists in our table of metaclusters (mapped to our existing `DpcStructMcsProperty` model), we display some predefined tables, as shown in Figure 2:

2.2.1 Final Result: Explorable tables



The screenshot shows a web browser at localhost:8000/dpctruct/mcs/MC0/#sequences. The page title is "DPCFam & DPCStruct Unified Database". The main content area is titled "DPCStruct | MC0" and has tabs for "Summary", "Sequences", "3D Viewer", and "Downloads". The "Sequences" tab is active, displaying a table of sequences. The table has columns: "Protein ID", "Range", and "Amino Acids (Representations)". The table lists 20 sequences, mostly with "None" in the "Amino Acids" column, except for the first two which have long amino acid strings. The table is paginated, showing "Page 1 of 3".

Protein ID	Range	Amino Acids (Representations)
AD41322A1.7	4-118	LSFLPLTTSTFTLTKAGNAGNDSDQFSGPAGWVVEVYLPEDYVAPKRYVYSGWLYSTLPAVHCLVQTL
AD4337M05	3-112	KSTFDFALCLAAFFHSGAQSTNEDQDQFGLAGQVVEVYLPEDYVAPKRYVYSGWLYSTLPAVHCLVQTL
AD4170W07	2-107	None
AD432NV07	1-133	None
AD432NV02	3-113	None
AD432NV01	48-158	None
AD432SP02	219-328	None
AD4338V02	138-258	None
AD4323842	5-101	None
AD4181U03	39-155	None
AD4172D06	3-108	None
AD4172D05	9-109	None
AD4311N02	2-118	None
AD4323803	6-129	None
AD4323808	5-81	None
AD4148007	15-140	None
AD4323809	6-109	None
AD4148023	53-174	None
AD4323804	4-132	None
AD4323805	16-144	None

Figure 2: DPCStruct Metaclusters Tables : Paginated Sequences table.

2.3 Next Steps

1. I am completing the preprocessing of data related to CATH and SCOP in order to incorporate minimum information from them (Figure 3) in the dpcstruct app.

cath_query	dpc_model	dpc_target	q_range	f_range	qlen	flen	qcov	fcov	qlen	qlscore	flscore	qlen	qlscore	flscore	qlen	qlscore
0	1.18.390	MC50	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
1	1.10.45	MC79	ADA512D1B.79	2-38	26-62	38	62	0.97	0.60	0.99	0.95	0.95	0.96	0.95	0.96	0.95
2	3.30.280	MC79	ADA439F1E.79	1-100	1-100	100	100	0.96	0.96	0.99	0.95	0.95	0.96	0.95	0.96	0.95
3	3.30.150	MC35	ADA7Y5MQR.93	1-128	1-114	143	202	0.90	0.56	1.29	0.60	0.45	0.45	0.61	1.16	0.56
4	2.30.80	MC35	ADA769DNR.95	2-208	400-600	251	601	0.82	0.33	235	0.58	0.28	0.28	0.63	1.13	0.33
5	2.30.200	MC35	ADA769DNR.95	7-102	2-244	263	637	0.97	0.36	256	0.66	0.36	0.36	0.70	23.2	0.36
6	3.40.220	MC124	ADA7P7M4A.124	3-108	13-140	128	223	0.96	0.57	142	0.57	0.36	0.36	0.50	11.2	0.57
7	1.10.155	MC124	ADA7P7M4A.124	7-75	2-71	75	266	0.92	0.26	70	0.78	0.24	0.24	0.73	24.2	0.26
8	3.30.1710	MC124	ADA7P7M4A.124	19-91	114-198	91	198	0.80	0.43	89	0.50	0.27	0.27	0.46	14.6	0.43
9	1.10.400	MC12	ADA7P7M4A.12	3-144	2-142	143	343	0.96	0.99	344	0.94	0.97	0.97	0.90	51.7	0.96

(a) CATH

scop_query	dpc_model	dpc_target	q_range	f_range	qlen	flen	qcov	fcov	qlen	qlscore	flscore	qlen	qlscore	flscore	qlen	qlscore
0	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
1	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
2	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
3	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
4	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
5	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
6	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
7	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
8	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52
9	Q2YQ20_2011190	MC79	ADA722D1A.50	1-78	64-141	78	100	0.52	78	0.60	0.52	0.52	0.60	0.52	0.60	0.52

(b) SCOP

Figure 3: Comparison with CATH and SCOP.

2. We have **1, 642,061** domains distributed over **1,252,179** proteins, organised into **28,246** metaclusters. In order to display how metaclusters overlap (if any) across the protein, we must know the total length of that protein. However, in contrast to dpcfam, we weren't given such data on protein lengths. What if we filter our new protein IDs in the existing dataframe (Figure 4)?:

```

1 # We use the DPCFam dataframe with info about proteins in UniProtKB
2 df_proteins = pd.read_csv("/u/mdmc/enyanduk/internship_areasciencepark/Dataframes/DPCFam/uniref50_proteins.csv")
3 df_proteins.head()

```

uniref50_id	uniprotkb_id	uniprotkb_accession	length
0	Q8WZ42-5	Q8WZ42-5	32900
1	Q3ASY8	Q3ASY8_CHLCH	36805
2	G580U1	G580U1_HETGA	36507
3	Q8WZ42	TITIN_HUMAN	34350
4	K7EE71	K7EE71_ORNAN	7472

Figure 4: For dpcfam, to display overlapping graphics, we were using protein IDs from the first column and their corresponding lengths from the last column.

By filtering dpcstruct (protein IDs) in the first column (Identifier in Uniref50) of Figure 4, **410,749** IDs are available (meaning the length is available) : **≈33%** of the total. Filtering in the third column (UniprotKB Accession Number), only **315, 482** are available: **≈25%** of the total. A sample of proteins missing in UniprotKB is shown below:

```

1 # Display some samples of df3["protein_id"] not in df_proteins["uniprotkb_accession"]
2 protein_ids_not_in_uniprotkb = df3[~df3["protein_id"].isin(unique_uniprotkb_accessions)]
3 print(protein_ids_not_in_uniprotkb.head())

```

mc_id	protein_id	prot_range
0	MC0	A0A1Q3ZAL7 4-118
1	MC0	A0A5371HW5 3-112
3	MC0	A0A3E2NVG7 1-133
4	MC0	A0A2W4V3S2 3-113
5	MC0	A0A4R2EPK1 49-158

Figure 5: Proteins not available in **UniProtKB**.

After a discussion, we agreed to write a script to fetch data from Uniprot. Please check out [fetch_uniprot_complete.py](#). I am still trying to figure out why the process is failing after 7 iterations (Figure 6):

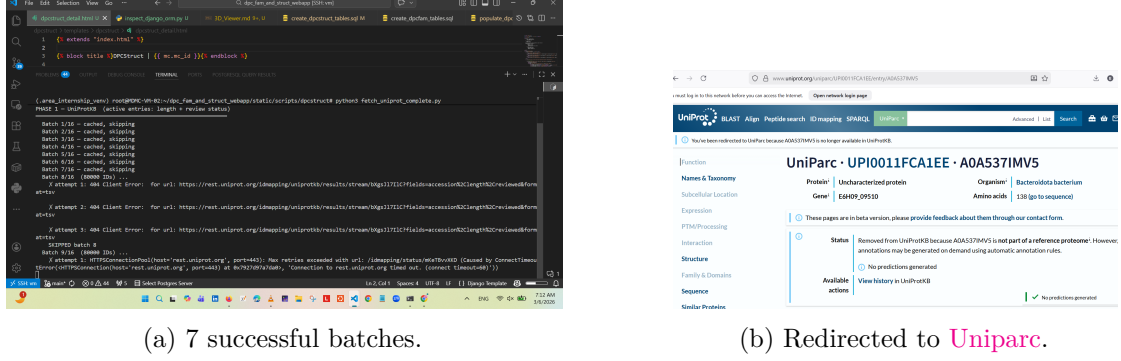


Figure 6: Fetching [Uniprot](#) Data failed.

3. Every single effort is being made in order to come up with an optimal way of displaying the 3D structures of representative PDB files for each metacluster, using the [Mol*](#) toolkit.

3 Conclusion

In order to display the current state of the project, please check out the [README.md](#) file. CSV files are available in [Google Drive](#). In addition to the three ongoing steps suggested above, I think we may also merge all metaclusters from both **Standard DPCfam** and **DPCfamB**, keeping in mind that for DPCfamB, we will miss only the AlphaFold representatives, as is already the case for some samples in Standard DPCfam (e.g. MC15). At the end of the day, we could have a complete list of metaclusters by merging the two disjoint and complementary DPCfam datasets.

References

- Barone, F., Laio, A., Punta, M., Cozzini, S., Ansuini, A., and Cazzaniga, A. (2025). Unsupervised domain classification of alphafold2-predicted protein structures. *PRX Life*, 3(2):023009.